A Project Report on

Paw Connect

By

Vikas Manik Mhetre

(PRN: 20240201124)

MCA – I, SEM – I

2024-25

To

Savitribai Phule Pune University,Pune

In Partial Fulfillment of the Degree of
Master in Computer Application (M. C. A.)

Under The Guidance Of
Prof. Neha Dhakol

Suryadatta Group of Institutes, Pune
Suryadatta Institute of Management and Mass Communication (SIMMC)

Date:-

CERTIFICATE

This is to certify that Mr. Vikas Manik Mhetre, has successfully completed his project work entitled **"Paw Connect"** in partial fulfillment of MCA – I Semester-I program for the year A.Y. 2024-25 . He have worked under our guidance and direction.

Prof.Neha Dhakol
Project Guide

HoD-MCA,SGI

Examiner 1

Examiner 2

Date :
Place :

Acknowledgment

We are the student of MCA first year. Here by we express our thanks to our project guide for allowing us to do the project on Paw Connect. This project work has been the most exciting part of our learning experience which would be an asset for our future carrier. We would especially like to thank our guide and mentor Prof. Neha Dhakol, who constantly guided us in developing, pushing us to search for more answers to her numerous questions. Also I would like to thank Dr. Rupali Dahake , project coordinators for her support. As a building block of MCA Department, I thank Dr. Manisha Kumbhar, HOD, MCA Department for her continuous support and help. We are grateful to many classmates who contributed their suggestions. Their hard work and examples push us to limits of our capability and encourage us daily.

Thank You

Vikas Manik Mhetre
(Seat No: 4834)

Index

Chapter	Page number
CHAPTER 1 : INTRODUCTION	
1.1 Existing System	**1-2**
1.2 Need for System	**2-3**
1.3 Operating Environment Hardware and Software	**4**
1.4 Brief Description about Technology Used	**5-6**
CHAPTER 2 : PROPOSED SYSTEM	
2.1 Proposed System (Introduction of system)	**7**
2.2 Module specifications (Scope)	**8**
2.3 Objectives of System	**9**
CHAPTER 3 : ANALYSIS & DESIGN	
3.1 Use Case Diagrams	**10**
3.2 Activity Diagram	**11**
3.3. Sequence Diagram	**12**
3.4 Class Diagram	**13**
3.5 Module Hierarchy Diagram	**14**
3.6 Table specifications (Database design)	**15-16**
3.7 Data dictionary	**17**
CHAPTER 4 : USER MANUAL	
4.1 User Interface Screens (Input)	**18-21**
4.2 Output Screens with data	**21-27**
4.3 Data Reports	**28-29**
4.4 Sample program code	**29-41**
4.5 Limitations	**42**
Bibliography	**43**

CHAPTER 1: INTRODUCTION

1.1 Existing System

The current systems employed by pet dog clinics for managing operations and scheduling appointments are often fragmented, inefficient, or entirely manual. These existing systems generally rely on outdated processes, such as:

1. **Paper-Based Systems**: Many clinics still utilize physical logbooks and paper forms to record appointments, medical histories, and prescriptions. This approach is prone to errors, misplacement of documents, and inefficiencies in accessing records.
2. **Phone-Based Appointment Scheduling**: Pet owners frequently call clinics to book appointments, leading to long wait times, scheduling conflicts, and a lack of transparency in available slots. This method also places a significant administrative burden on clinic staff.
3. **Standalone Software Solutions**: Some clinics use basic software applications that are not tailored for veterinary operations. These solutions often lack critical features such as integrated appointment history tracking, prescription management, or compatibility with multiple devices and browsers.
4. **Limited Communication with Pet Owners**: Existing systems do not effectively engage pet owners through online content, such as blogs or newsletters, which could educate them on dog care and clinic services.
5. **No Centralized Database**: Without a robust database system, accessing and managing pet records, doctor schedules, and clinic statistics becomes cumbersome. Clinics may rely on fragmented systems that do not communicate with one another, causing inefficiencies.
6. **Poor Data Security**: Physical records or standalone software solutions often lack proper security measures, making sensitive data susceptible to loss, theft, or unauthorized access.
7. **Lack of Online Presence**: Many clinics fail to have a significant online presence, making it difficult for pet owners to discover their services or learn about their expertise. This gap results in missed opportunities to build trust and attract new clients.

The shortcomings of these systems hinder clinic efficiency, reduce customer satisfaction, and limit the ability to scale operations effectively. Clinics operating with such systems face challenges in maintaining accurate records, improving patient outcomes, and meeting the expectations of modern pet owners accustomed to digital conveniences.

1.2 Need for System

The "Paw Connect" system addresses the critical need for a comprehensive, user-friendly, and efficient platform tailored specifically for pet dog clinics. The following points highlight the necessity of this system:

1. **Streamlined Operations**: By digitizing appointment scheduling, record management, and prescription tracking, Paw Connect eliminates the inefficiencies of paper-based and fragmented systems. This streamlining allows clinic staff to focus on providing quality care instead of administrative tasks.
2. **Improved Customer Experience**: Pet owners benefit from a seamless and intuitive platform where they can book appointments, view their pets' medical history, and stay informed through blogs and articles. These features enhance the overall experience and build long-term loyalty.
3. **Centralized Data Management**: The use of a MySQL database ensures that all data related to appointments, prescriptions, and clinic operations is stored securely in one place. This centralization makes data retrieval and analysis easier for both users and administrators.
4. **Enhanced Accessibility**: With compatibility across major web browsers like Chrome, Firefox, and Safari, Paw Connect ensures that users can access the platform on any device, at any time. This accessibility aligns with the expectations of modern users.
5. **Tailored Features for Clinics**:
    - **Admin Dashboard**: Provides administrators with real-time insights into clinic performance, appointment statistics, and operational details.
    - **Prescription Management**: Allows both users and administrators to add and manage prescriptions linked to specific appointments, improving medical record accuracy.
    - **Team Information**: Highlights the expertise of the clinic's staff, fostering trust among pet owners.

6. **Improved Communication**: The inclusion of a blog feature allows clinics to share valuable information about dog care, health tips, and clinic updates. This not only educates pet owners but also positions the clinic as an authoritative resource in the industry.
7. **Scalability and Future Growth**: With a robust backend powered by Python Flask and PHP, Paw Connect is designed to scale as clinics expand their operations or introduce new services. The platform's architecture supports the addition of features without disrupting existing functionalities.
8. **Data Security and Compliance**: By utilizing secure database management practices and modern web technologies, Paw Connect ensures the confidentiality and integrity of sensitive information. This focus on security builds trust among users and complies with data protection regulations.
9. **Market Competitiveness**: As pet owners increasingly seek digital solutions for managing their pets' healthcare, clinics that adopt innovative platforms like Paw Connect gain a competitive edge. The system's modern approach aligns with the expectations of tech-savvy customers.
10. **Enhanced Efficiency for Administrators**: Features like fetching appointments by date and managing prescriptions reduce the workload on administrators, allowing them to focus on strategic decisions rather than routine tasks.

In summary, Paw Connect is an essential solution for modernizing pet dog clinic operations. Its comprehensive feature set, focus on user experience, and commitment to efficiency address the limitations of existing systems while meeting the evolving needs of pet owners and clinics alike.

1.3 Operating Environment: Hardware and Software

Hardware Requirements:

- **Client Devices:** Desktops, laptops, tablets, or smartphones.
- **Server Requirements:** A server capable of running PHP scripts and Python Flask applications with at least 4 GB of RAM and 500 GB of storage.
- **Network:** Stable internet connection for seamless data exchange.

Software Requirements:

- **Frontend:** HTML, CSS, and Bootstrap for responsive design.
- **Backend:** Python Flask for backend logic and PHP for server-side scripting.
- **Database:** MySQL for storing user and appointment data.
- **Browser Compatibility:** Chrome, Firefox, Safari.
- **Operating System:** Compatible with major OS platforms such as Windows, macOS, Linux, Android, and iOS.

1.4 Brief Description of Technology Used

The Paw Connect platform leverages a combination of modern technologies to deliver a comprehensive and user-friendly solution for pet dog clinics. Below is an overview of the technologies employed:

1. **Frontend Development**:
    - **HTML and CSS**: These core web technologies form the foundation of the platform's design, structure, and styling, ensuring a clean and visually appealing user interface.
    - **Bootstrap**: This framework enhances the responsiveness of the platform, enabling it to adapt seamlessly to different screen sizes and devices. Bootstrap's components and utilities accelerate the development process while maintaining design consistency.
2. **Backend Development**:
    - **Python Flask**: Flask serves as the primary backend framework, handling application logic, routing, and integration with the database. Its lightweight and flexible nature makes it ideal for building scalable and efficient web applications.
    - **PHP**: PHP is used for server-side scripting, enabling dynamic content generation and ensuring smooth communication between the server and the client.
3. **Database Management**:
    - **MySQL**: A robust relational database system is employed for storing and managing data related to appointments, user information, prescriptions, and clinic statistics. MySQL's reliability and performance make it a popular choice for web applications.
4. **Browser Compatibility**:
    - Paw Connect is designed to function seamlessly across major web browsers, including **Google Chrome**, **Mozilla Firefox**, and **Safari**. This ensures that users can access the platform without encountering compatibility issues.
5. **Security Measures**:
    - The platform incorporates secure coding practices and database management to safeguard sensitive information, such as pet medical histories and user

credentials. SSL encryption and authentication mechanisms are used to enhance security.

6. **User-Friendly Features**:
    - The integration of interactive elements and intuitive navigation ensures a positive user experience for both pet owners and administrators. Features such as appointment scheduling, prescription management, and blog access are designed for ease of use.

By combining these technologies, Paw Connect delivers a robust, scalable, and efficient solution that meets the needs of modern pet dog clinics and their clients.

CHAPTER 2: PROPOSED SYSTEM

2.1 Proposed System

The "Paw Connect" system represents a comprehensive digital solution aimed at addressing the challenges faced by pet dog clinics in managing their operations. The proposed system is designed to streamline administrative tasks, enhance user engagement, and ensure seamless communication between clinic staff and pet owners.

Key highlights of the proposed system include:

1. **Digital Appointment Scheduling**: A user-friendly interface allows pet owners to book appointments conveniently, minimizing the administrative burden on clinic staff and reducing scheduling conflicts.
2. **Integrated Medical Records**: Pet owners and clinic staff can access a centralized database that stores comprehensive medical histories, prescriptions, and treatment details for each pet.
3. **Admin Dashboard**: The system provides a dedicated dashboard for administrators to monitor clinic performance, track appointments, and manage team activities efficiently.
4. **Interactive Blog Section**: A blog feature educates pet owners on various topics related to dog health, fostering community engagement and positioning the clinic as a trusted resource.
5. **Secure and Scalable Platform**: Built with robust technologies, the system ensures data security and scalability to support future growth and feature enhancements.

This proposed system bridges the gap between traditional clinic operations and the expectations of tech-savvy pet owners, offering a seamless and modern solution for both users and administrators.

2.2 Module Specifications

The scope of the Paw Connect system is defined through its core modules, each designed to address specific needs of pet dog clinics and their customers. The major modules include:

1. **User Module**:
    - **Book Appointment**: Allows pet owners to schedule appointments and view available time slots.
    - **Appointment History**: Displays past appointments, treatments, and prescriptions for easy reference.
    - **Add Prescription**: Enables users to upload prescriptions or medical notes linked to their appointments.
2. **Admin Module**:
    - **Manage Appointments**: Provides functionality to view and manage all appointments by date, ensuring smooth clinic operations.
    - **Add Prescriptions**: Allows administrators to update or add prescriptions to individual appointments for better record-keeping.
    - **Admin Dashboard**: Offers real-time insights into clinic statistics, performance metrics, and staff activities.
3. **Content Module**:
    - **Blog Management**: Hosts educational and informative content about pet care, increasing user engagement.
    - **Team Information**: Highlights the expertise and qualifications of clinic staff to build user trust.
4. **Database Management Module**:
    - Ensures secure storage and retrieval of data, including user details, appointments, and medical records.
5. **Security Module**:
    - Implements authentication and encryption protocols to protect sensitive information.

Each module is designed to integrate seamlessly with the others, providing a holistic and efficient solution for clinic operations.

2.3 Objectives of the System

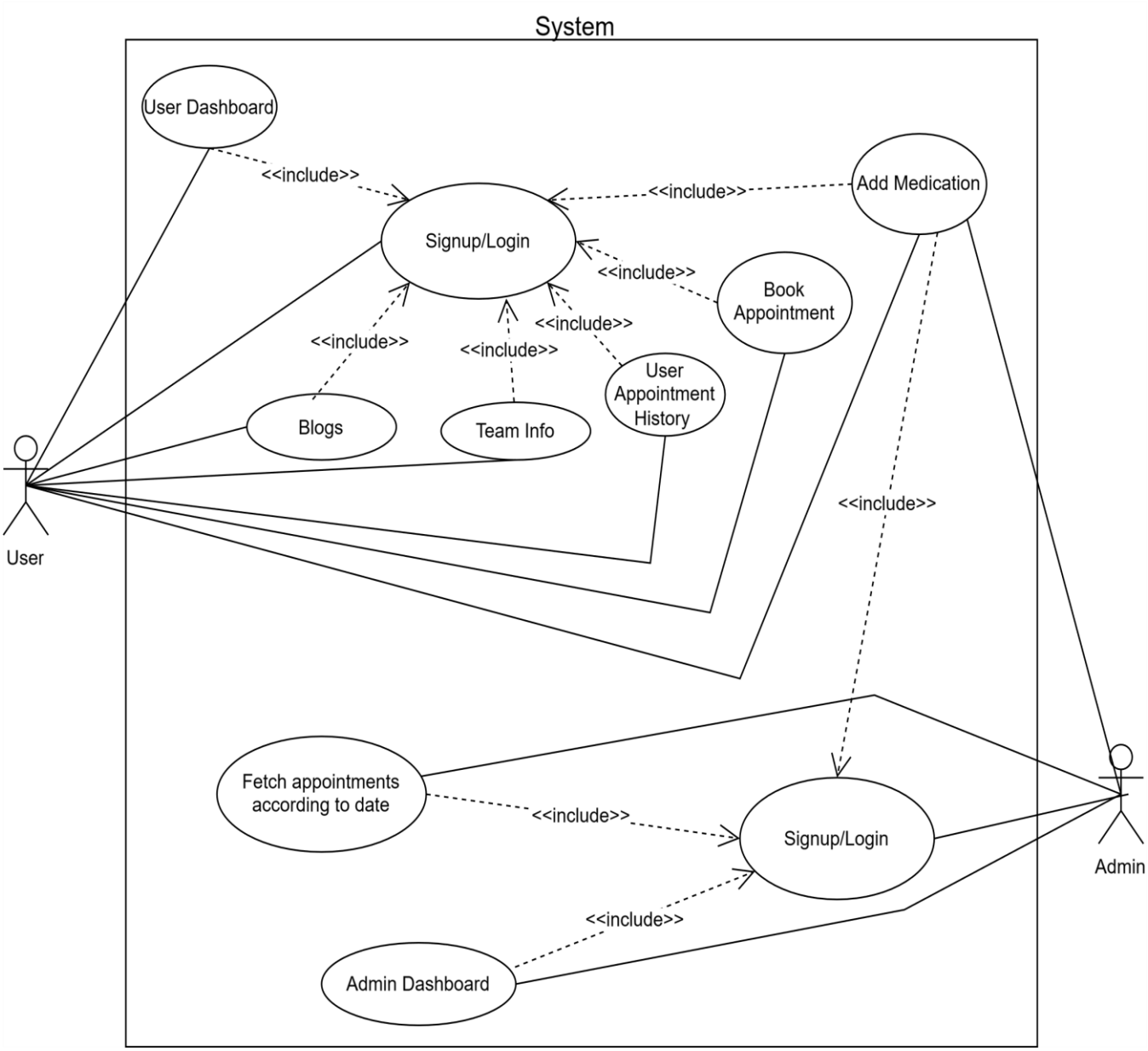
The primary objectives of the Paw Connect system are as follows:

1. **Streamline Clinic Operations**:
    - Automate administrative tasks like appointment scheduling and prescription management, reducing workload and minimizing errors.
2. **Enhance User Experience**:
    - Provide pet owners with an intuitive platform to manage appointments, access medical histories, and stay informed through blogs and updates.
3. **Centralize Data Management**:
    - Maintain a secure and centralized database for all clinic-related information, enabling easy access and efficient record-keeping.
4. **Improve Communication**:
    - Facilitate seamless interaction between pet owners and clinic staff through real-time updates and notifications.
5. **Support Scalability**:
    - Design the system to accommodate future growth, including new features, increased user base, and additional clinics.
6. **Ensure Data Security**:
    - Employ robust security measures to protect sensitive information and build user trust.
7. **Promote Pet Care Awareness**:
    - Use the blog feature to educate pet owners about dog health, treatments, and preventive care, fostering a knowledgeable and engaged community.

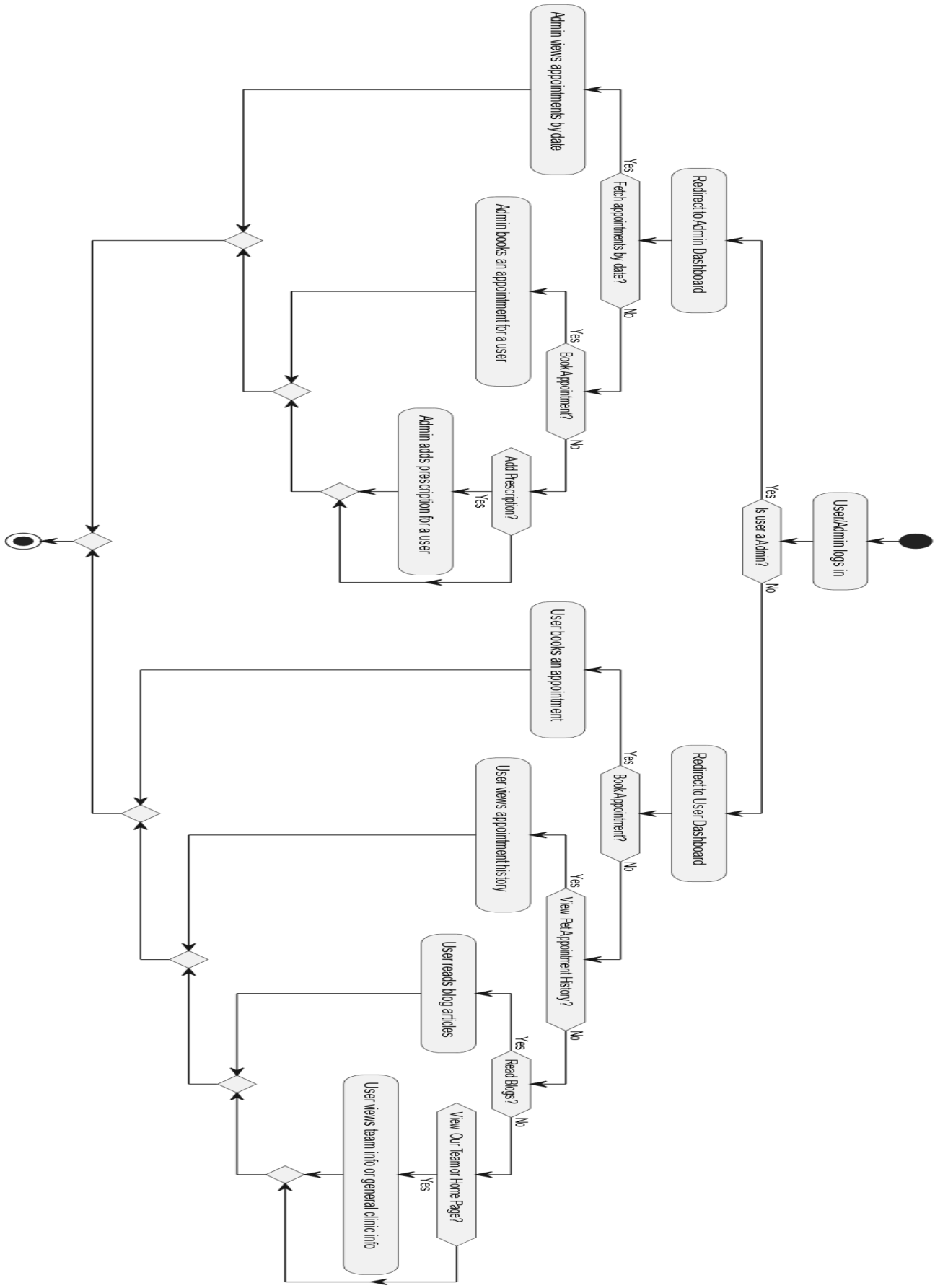
By achieving these objectives, the Paw Connect system aims to transform the way pet dog clinics operate, delivering enhanced value to both users and administrators.

CHAPTER 3 : ANALYSIS & DESIGN

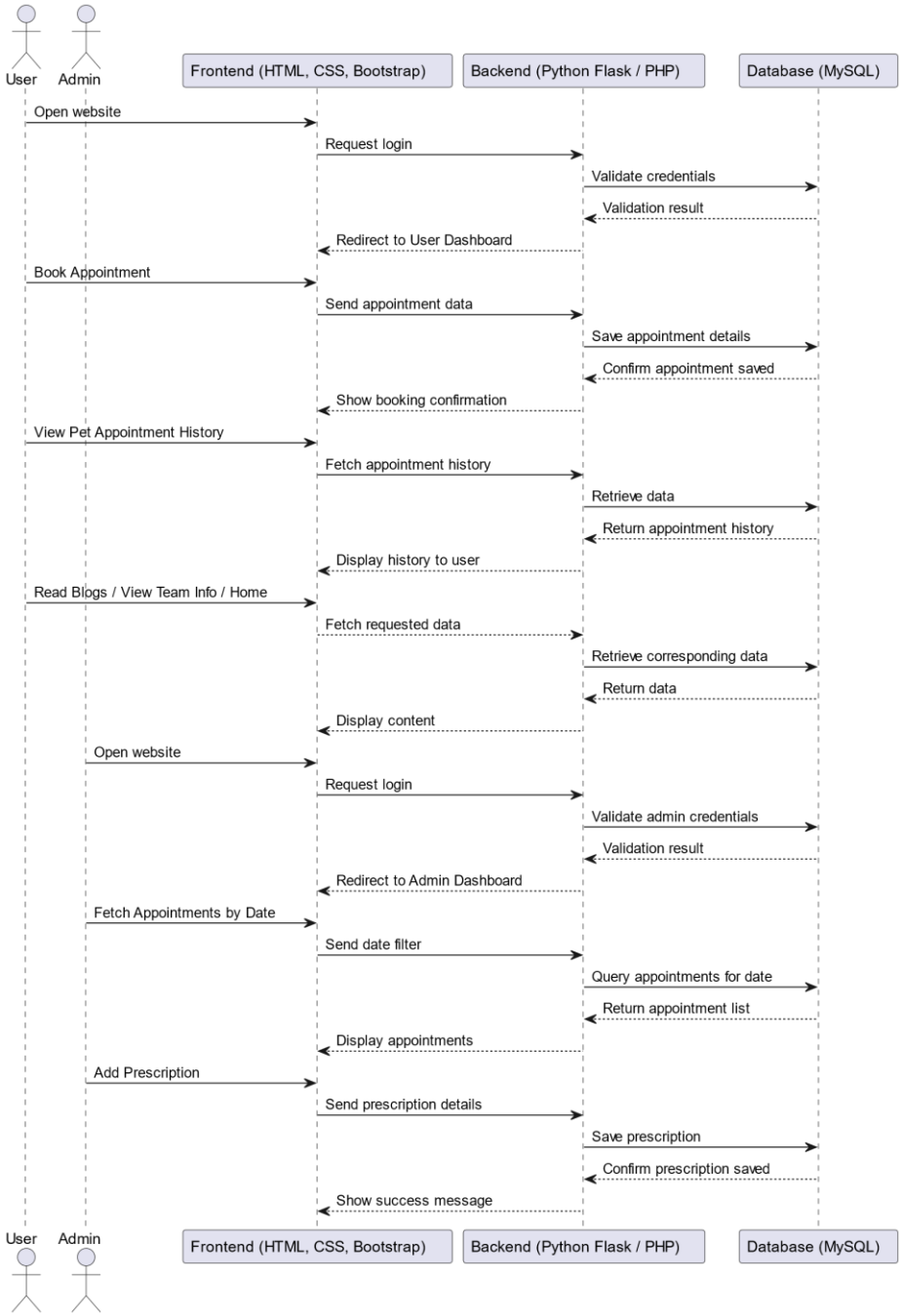
3.1 Use-Case Diagram:



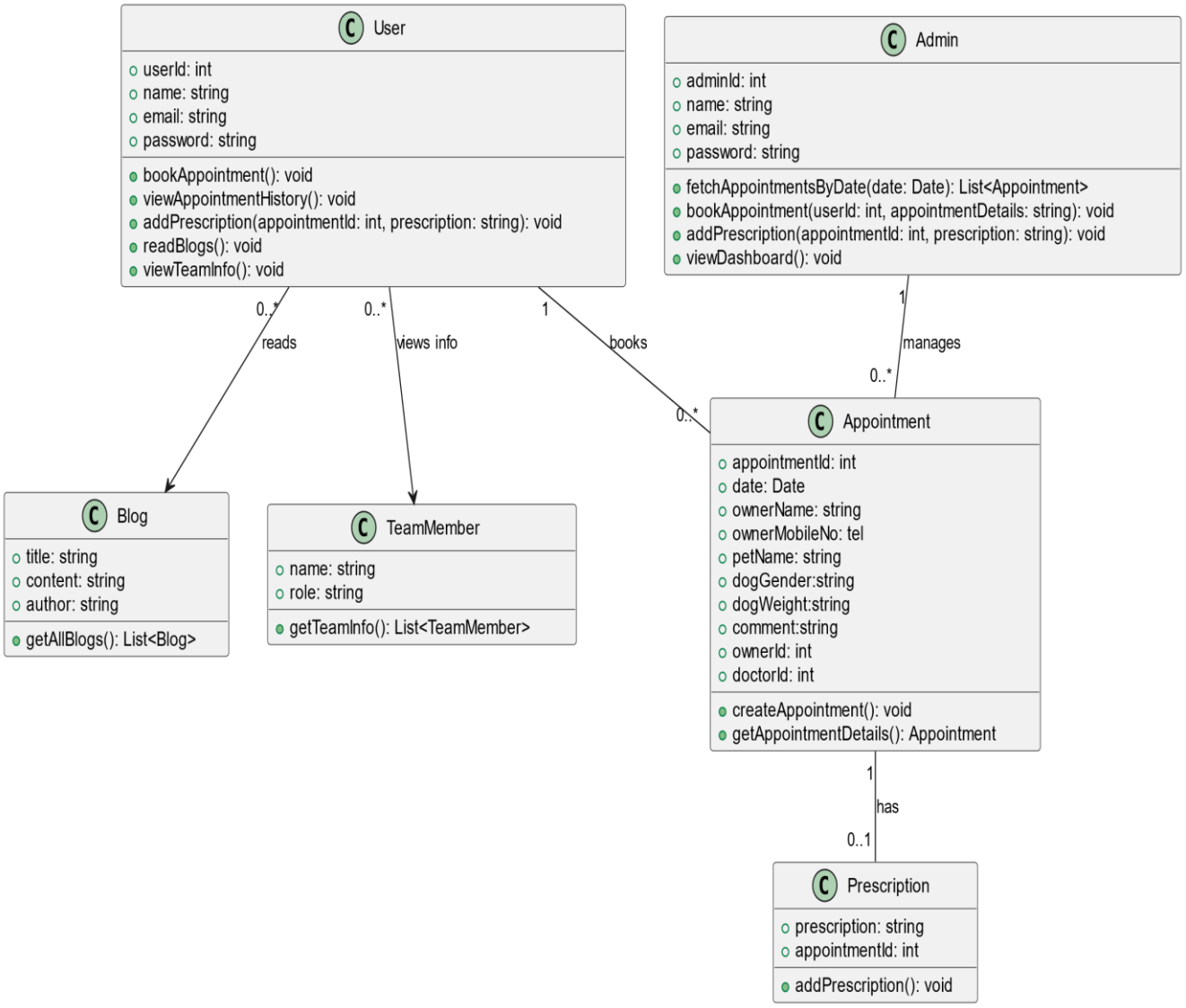
3.2 Activity Diagram:



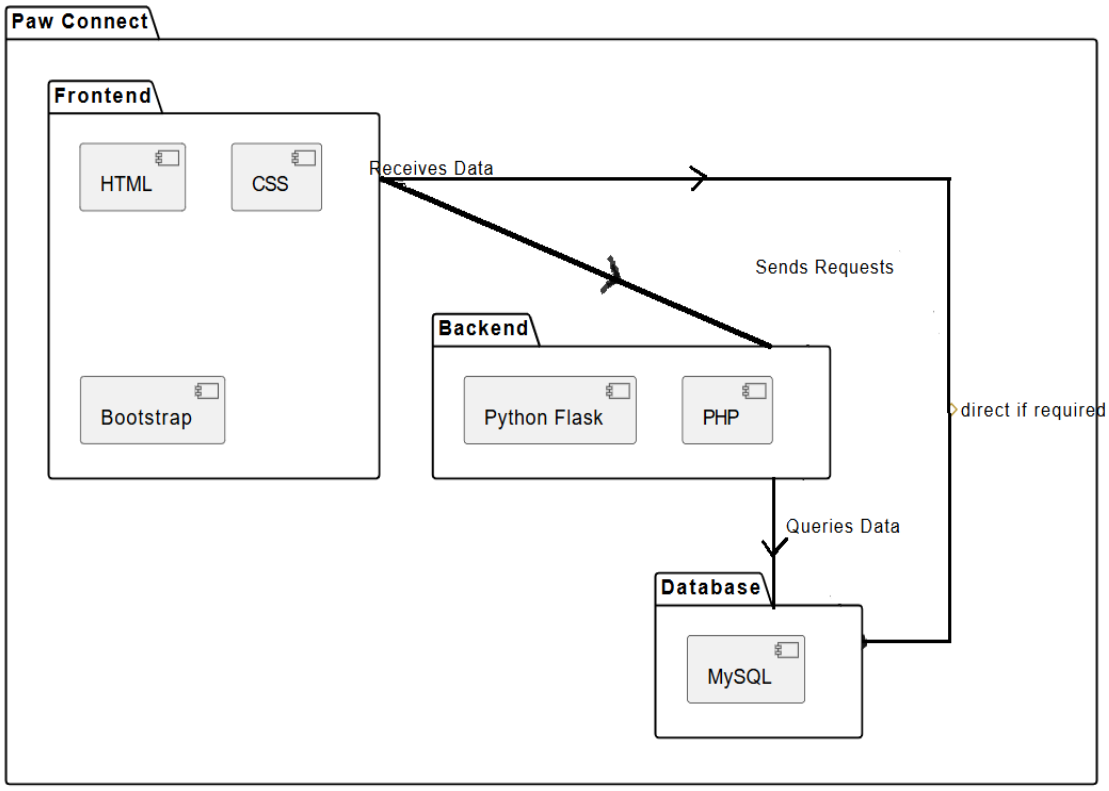
3.3. Sequence Diagram:



3.4 Class Diagram:



3.5 Module Hierarchy Diagram:



3.6 Table specifications (Database design):

1. admin_users table:

- **Purpose**: Store ADMIN information for authentication
- **Fields**:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**id**	int(10)			No	*None*		AUTO_INCREMENT	Change Drop More
2	**name**	varchar(100)	utf8mb4_general_ci		No	*None*			Change Drop More
3	**mobile**	int(100)			No	*None*			Change Drop More
4	**password**	varchar(100)	utf8mb4_general_ci		No	*None*			Change Drop More
5	**email**	varchar(100)	utf8mb4_general_ci		No	*None*			Change Drop More

2. users:

- **Purpose**: Store USER information for authentication.
- **Fields:**

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**id**	int(10)			No	*None*		AUTO_INCREMENT	Change Drop More
2	**name**	varchar(100)	utf8mb4_general_ci		No	*None*			Change Drop More
3	**mobile**	int(100)			No	*None*			Change Drop More
4	**password**	varchar(20)	utf8mb4_general_ci		No	*None*			Change Drop More
5	**email**	varchar(100)	utf8mb4_general_ci		No	*None*			Change Drop More

Check all *With selected:* Browse Change Drop Primary Unique Index Spatial Fulltex

3. appointment_YYYY_MM_DDDD Table:

- **Purpose**: This table is created dynamically, propose is to book appointment according to date and validate appointment Only 10 appointment per date.

- **Fields:**

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**id**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
2	**owner_name**	varchar(255)	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
3	**owner_tel**	varchar(20)	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
4	**reason_to_book**	text	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
5	**dog_name**	varchar(255)	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
6	**dog_gender**	varchar(10)	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
7	**dog_weight**	float			Yes	*NULL*			Change Drop More
8	**date_of_appointment**	date			Yes	*NULL*			Change Drop More
9	**explain_reason**	text	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
10	**email_apt**	varchar(255)	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
11	**pass_apt**	varchar(255)	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
12	**medication**	text	utf8mb4_general_ci		Yes	*NULL*			Change Drop More

Check all *With selected:* Browse Change Drop Primary Unique Index Spatial Fulltext

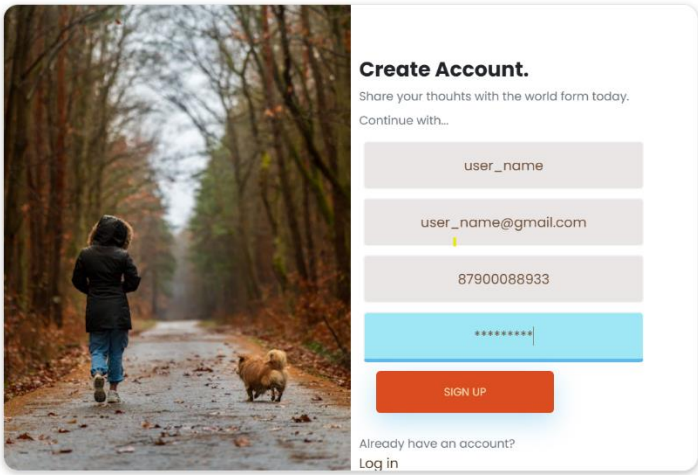
3.7 Data dictionary

Field Name	Data Type	Description	Table Name
id(for admin)	Int	Unique identifier for the admin.	admin_users
id(for User account)	Int	Reference to the userId of the user.	users
email	Varchar	Reference to the email Id of the user.	users
password	Varchar	Reference to the password of the user email.	users
date_of_appointment	date	Date and time the appointment was added to appointments	appointment_yyyy _mm_dd
Id(Record id in appointments)	Int	Unique record d id created and reference to date of appointment for further modification	appointment_yyyy _mm_dd
owner_name	string	Reference to owner name who book appointment	appointment_yyyy _mm_dd
dog_name	string	Reference to dog name for whobook appointment	appointment_yyyy _mm_dd
dog_gender	string	Reference to dog gender	appointment_yyyy _mm_dd
email_apt	string	Reference To fatch user appointments	appointment_yyyy _mm_dd
pass_apt	string	Reference To fatch user appointments	appointment_yyyy _mm_dd

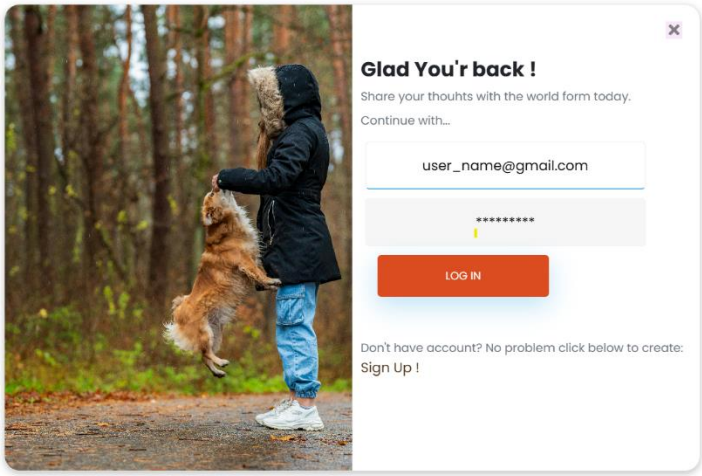
CHAPTER 4: USER MANUAL

4.1 User Interface Screens (Input):

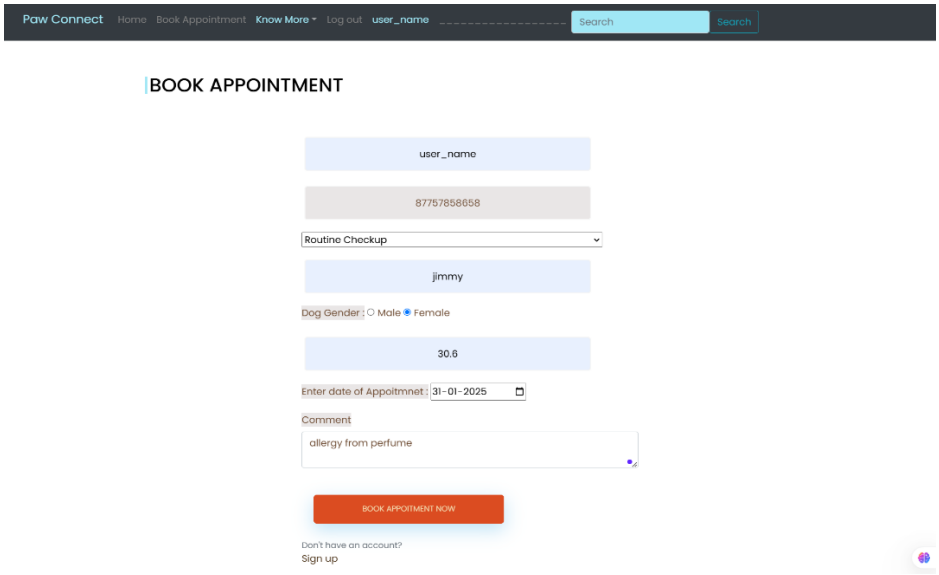
- User signup :



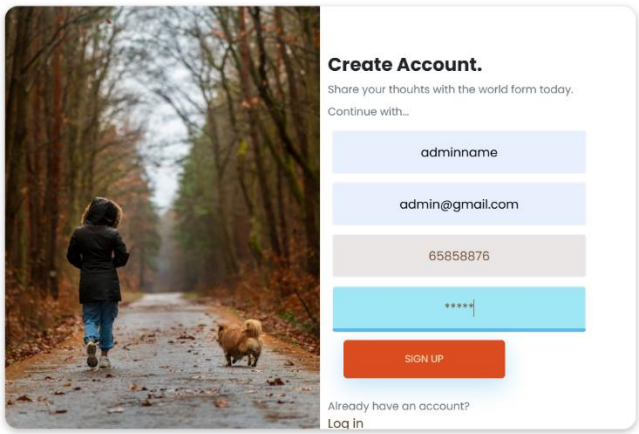
- User login:



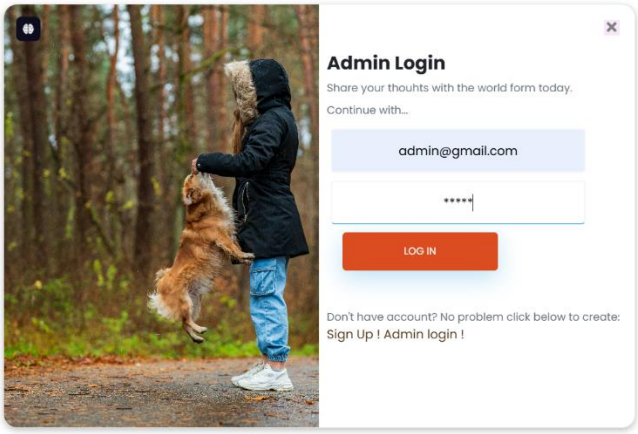
- User Book Appointment input:



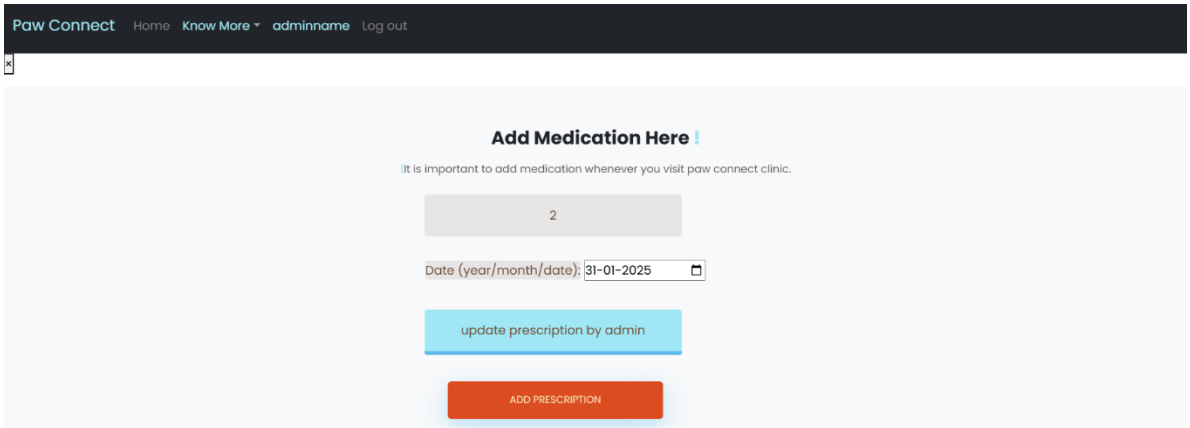
- Admin signup input:



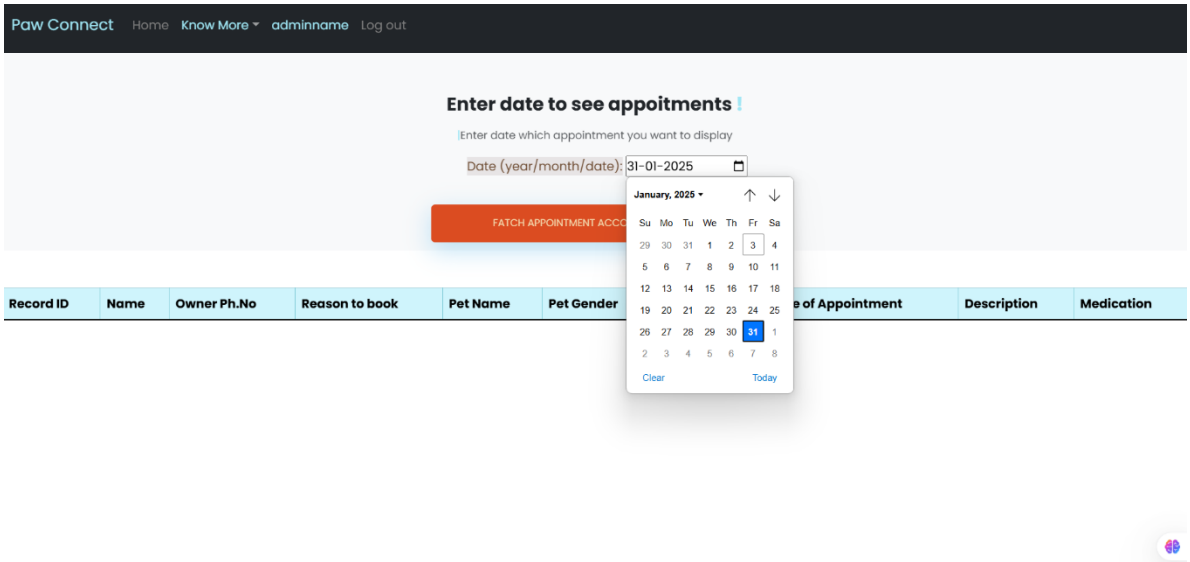
- Admin Login input:



- Admin add prescription:

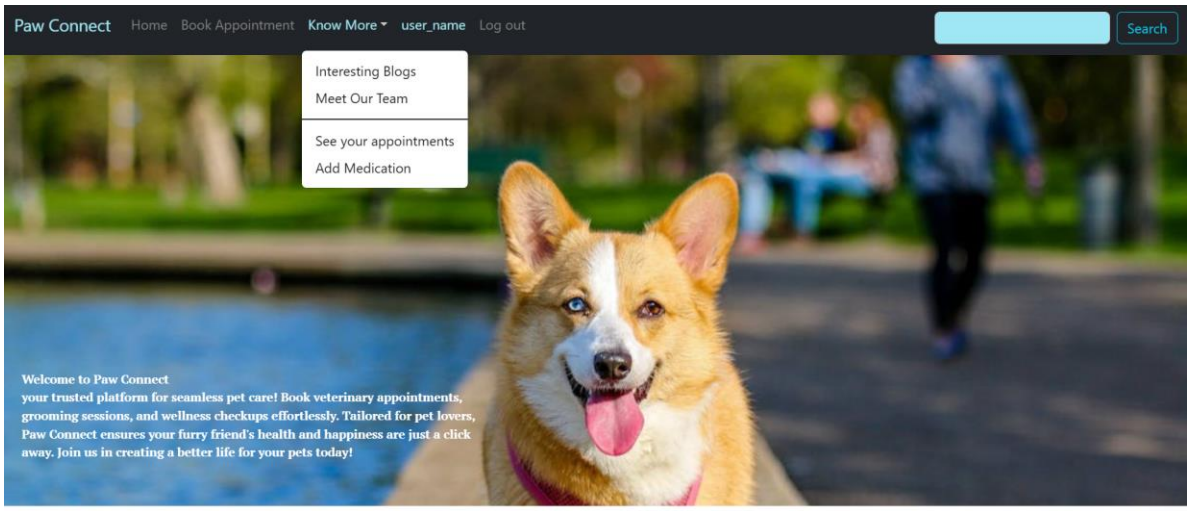


- Admin fetch appointment according to date input:

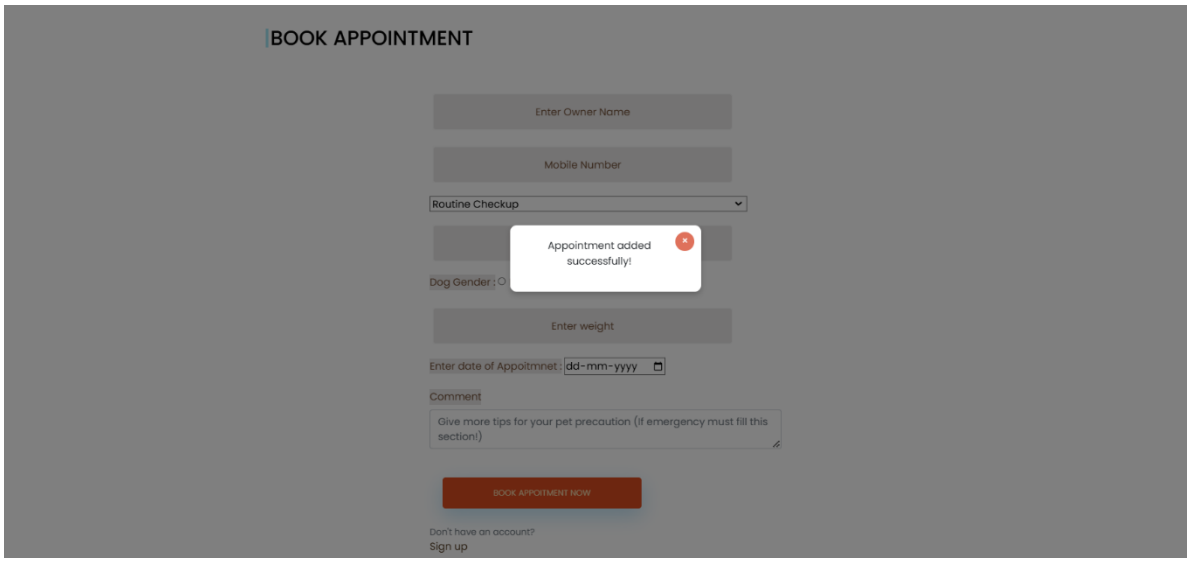


4.2 Output Screens with data:

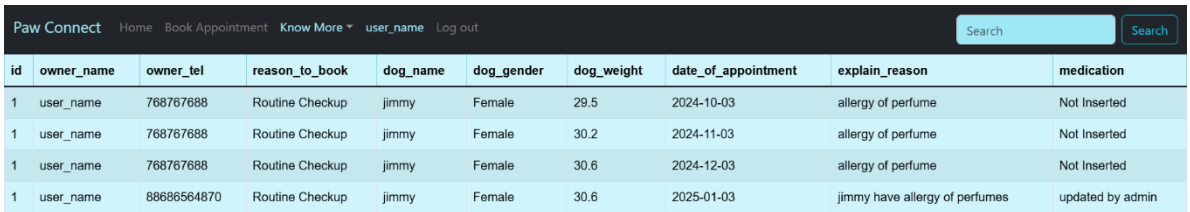
- User login:



- User Book Appointment Output:

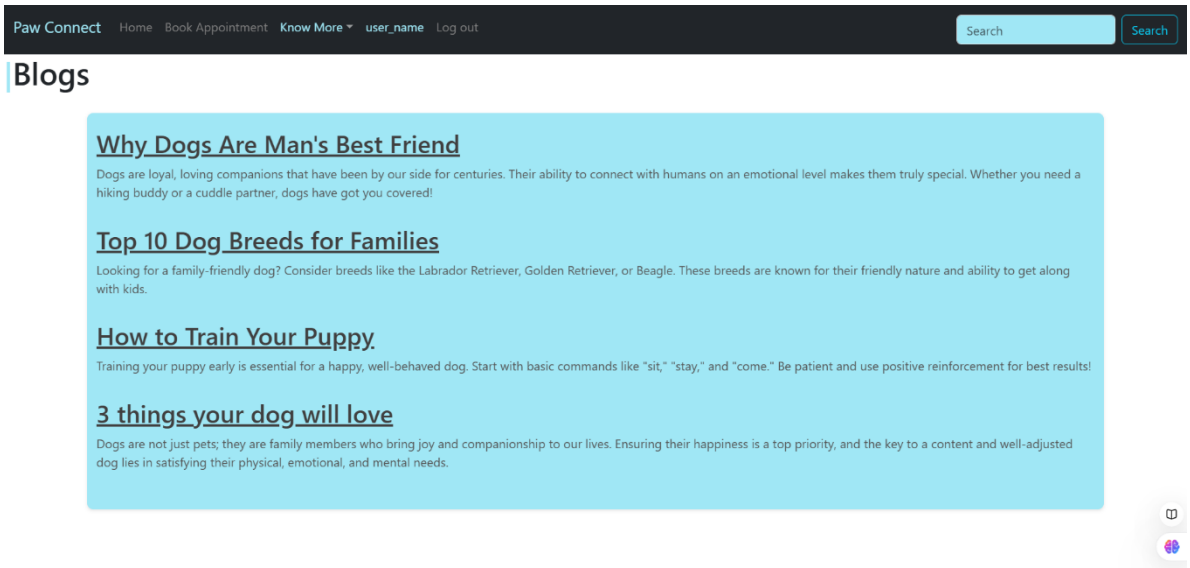


- User appointment history:



id	owner_name	owner_tel	reason_to_book	dog_name	dog_gender	dog_weight	date_of_appointment	explain_reason	medication
1	user_name	768767688	Routine Checkup	jimmy	Female	29.5	2024-10-03	allergy of perfume	Not Inserted
1	user_name	768767688	Routine Checkup	jimmy	Female	30.2	2024-11-03	allergy of perfume	Not Inserted
1	user_name	768767688	Routine Checkup	jimmy	Female	30.6	2024-12-03	allergy of perfume	Not Inserted
1	user_name	88686564870	Routine Checkup	jimmy	Female	30.6	2025-01-03	jimmy have allergy of perfumes	updated by admin

- User blog output 1:



- User blog output 2:

Paw Connect Home Book Appointment Know More user_name Log out Search Search

Why Dogs Are Man's Best Friend

Dogs are loyal, loving companions that have been by our side for centuries. Their ability to connect with humans on an emotional level makes them truly special. Whether you need a hiking buddy or a cuddle partner, dogs have got you covered! Why Dogs Are Man's Best Friend Dogs have earned the title of "man's best friend" for centuries, and it's no surprise why. These loyal, loving, and intuitive animals offer companionship, protection, and unconditional love. Their unique relationship with humans has stood the test of time, making them more than just pets—they're family. Here's why dogs have become our most beloved

1. Unconditional Love and Loyalty

One of the most remarkable traits of dogs is their unwavering loyalty. Regardless of circumstances, dogs remain devoted to their owners, forming deep, emotional bonds. They're always happy to see you, wagging their tails and greeting you enthusiastically at the door after a long day. Dogs don't judge; they simply love. Whether you're feeling happy, sad, or stressed, they offer comfort and affection. Their ability to sense emotions and respond accordingly is a testament to their intuitive nature. This unique ability makes them reliable emotional support companions, helping their owners through tough times.

2. Protective Instincts

Dogs have a natural instinct to protect those they care about. Many breeds are known for their guarding abilities, often putting themselves between their owners and potential danger. This protective nature makes dogs not only great companions but also trustworthy guardians. For families, a dog can be a source of safety, alerting you to strangers or unusual noises. Their protective instincts extend beyond physical threats; they often sense when something is wrong emotionally or mentally and stay by your side to offer support.

3. Health Benefits

Having a dog positively impacts physical and mental health. Walking your dog encourages regular exercise, improving cardiovascular health and reducing stress. Studies have shown that dog owners often have lower blood pressure and cholesterol levels compared to non-owners. Mentally, dogs help reduce anxiety and depression. Their presence can provide a calming effect, and interacting with them releases oxytocin, a hormone associated with happiness and bonding. Dogs also help combat loneliness, especially for those living alone or in challenging circumstances.

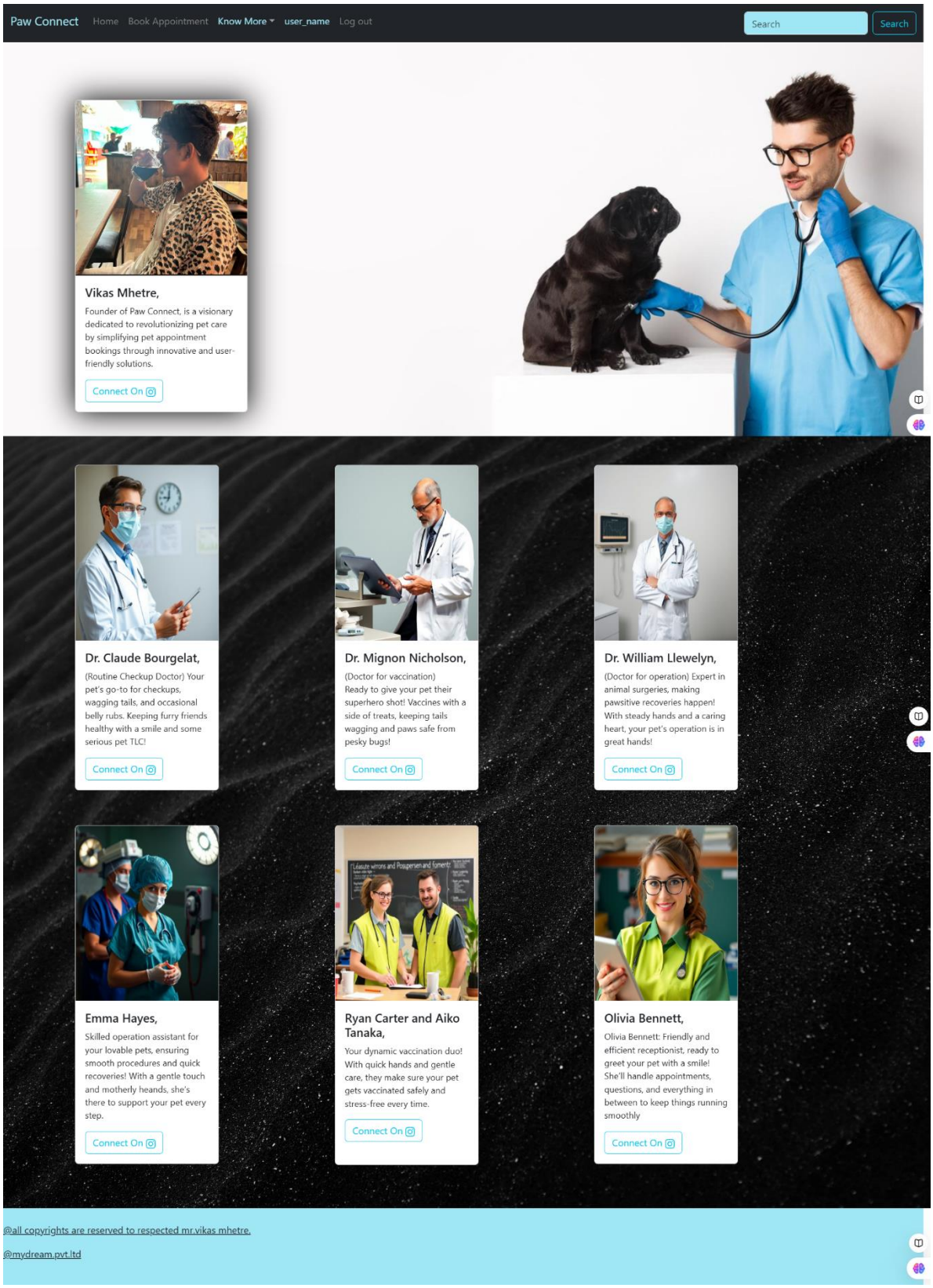
4. Unmatched Companionship

Dogs thrive on companionship and love being part of the family. Whether you're going for a walk, watching TV, or working from home, dogs want to be by your side. Their playful and affectionate nature brings joy and laughter to everyday life. They also teach us valuable lessons about living in the moment and appreciating the simple joys in life. A wagging tail or an excited bark reminds us to cherish the small things, like a sunny day or a quiet moment together.

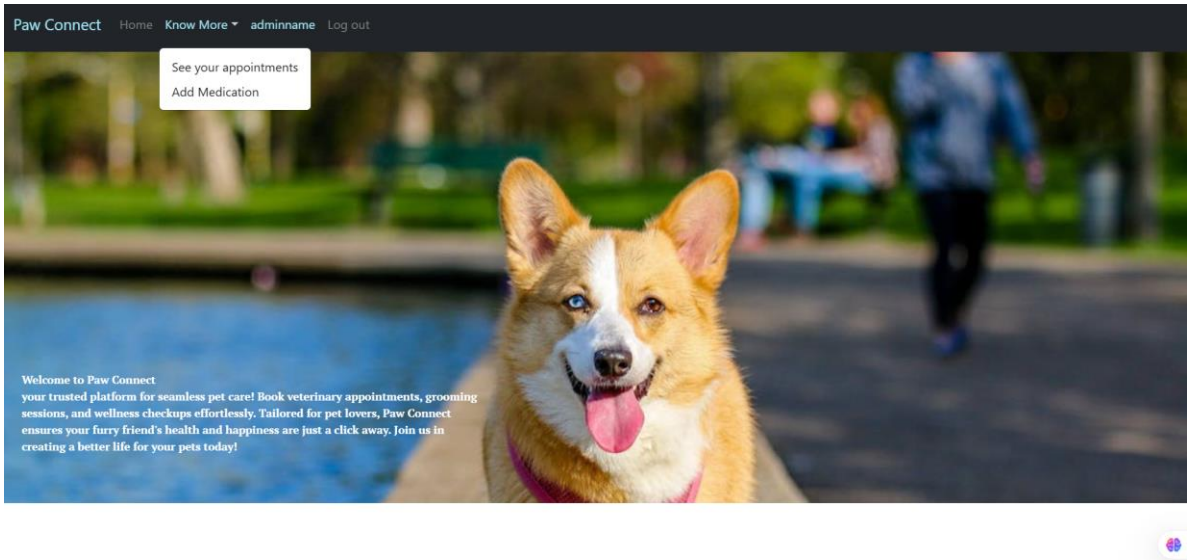
Conclusion:
Dogs are man's best friend because they embody loyalty, love, and companionship like no other animal. They protect us, improve our health, and bring unparalleled joy to our lives. This enduring bond between humans and dogs is built on mutual trust, affection, and an unspoken understanding that transcends words. A dog isn't just a pet—they're a best friend, a family member, and a source of endless happiness.

Top 10 Dog Breeds for Families | How to Train Your Puppy: A Step-by-Step Guide | 3 Things Your Dog Will Love

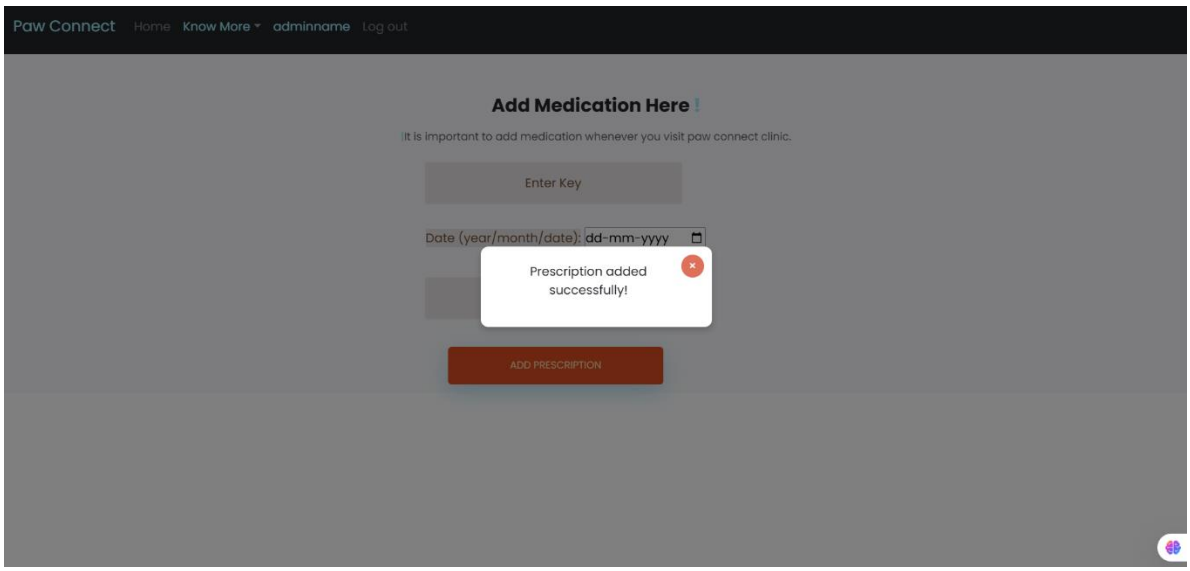
- User team info output:



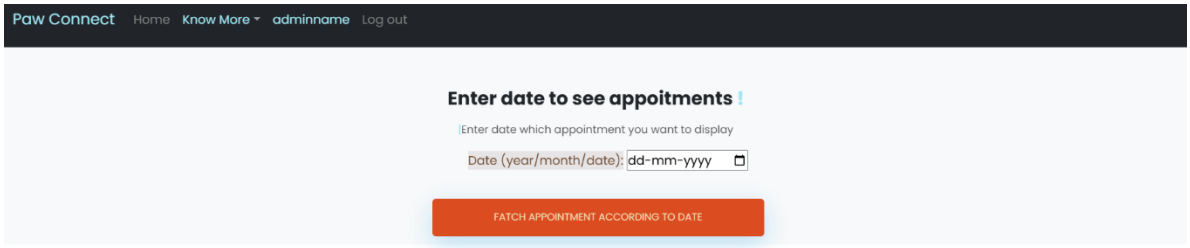
- Admin login output:



- Admin add prescription output:

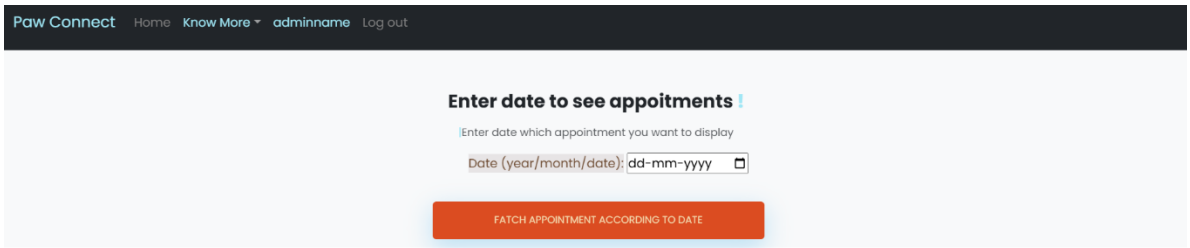


- Admin add prescription output 2:



Record ID	Name	Owner Ph.No	Reason to book	Pet Name	Pet Gender	pet Weight	Date of Appointment	Description	Medication
1	vikas	1	Routine Checkup	jimmy	Female	30.1	2025-01-31	cap1	updated by admin
2	rahul	2	Routine Checkup	maxy	Male	30.2	2025-01-31	cap2	update prescription by admin
3	raj	3	Vaccination	julle	Female	30.3	2025-01-31	cap3	Not Inserted

- Admin fetch appointment according to date output:



Record ID	Name	Owner Ph.No	Reason to book	Pet Name	Pet Gender	pet Weight	Date of Appointment	Description	Medication
1	user_name	88686564870	Routine Checkup	jimmy	Female	30.6	2025-01-03	jimmy have allergy of perfumes	updated by admin
2	rajesh	574653634	Vaccination	julle	Female	30.1	2025-01-03	no comments	Not Inserted
3	shreyash	62347923	Vaccination	mani	Male	20.0	2025-01-03	no comments	Not Inserted

4.3 Data Reports:

- Data report of date 03-12-2024:

	id	owner_name	owner_tel	reason_to_book	dog_name	dog_gender	dog_weight	date_of_appointment	explain_reason	email_apt	pass_apt	medication
Edit Copy Delete	1	user_name	768767688	Routine Checkup	jimmy	Female	30.6	2024-12-03	allergy of perfume	user_name@gmail.com	*****	Not Inserted
Edit Copy Delete	2	ramesh	856685856	Routine Checkup	browny	Male	30.6	2024-12-03	no comments	raj@gmail.com	rrrrrr	Not Inserted
Edit Copy Delete	3	manoj	767897978	Vaccination	swiggy	Female	30.5	2024-12-03	swiggy is paralised so take extra care at the time...	raj@gmail.com	rrrrrr	Not Inserted
Edit Copy Delete	4	sachin	87349837982	Routine Checkup	ammy	Female	28.7	2024-12-03	no comments	raj@gmail.com	rrrrrr	Not Inserted
Edit Copy Delete	5	praney	7397233	Operation	tizzy	Male	30.6	2024-12-03	operation done back in 24 dec 2023	raj@gmail.com	rrrrrr	Not Inserted
Edit Copy Delete	6	hitesh	67979723	Routine Checkup	superman	Male	30.5	2024-12-03	no comments	raj@gmail.com	rrrrrr	Not Inserted
Edit Copy Delete	7	ammay	867858758	Vaccination	sony	Female	30.3	2024-12-03	no comments	raj@gmail.com	rrrrrr	Not Inserted
Edit Copy Delete	8	Dinesh	97397223	Routine Checkup	happy	Male	40	2024-12-03	happy is taking medication from previous 1 month	raj@gmail.com	rrrrrr	Not Inserted
Edit Copy Delete	9	DK	938723982	Vaccination	Fog	Female	30.1	2024-12-03	no comments	raj@gmail.com	rrrrrr	Not Inserted
Edit Copy Delete	10	nakhil	73798232	Operation	iron	Male	40.5	2024-12-03	no comments	raj@gmail.com	rrrrrr	Not Inserted

- Data report of date 03-01-2025:

	id	owner_name	owner_tel	reason_to_book	dog_name	dog_gender	dog_weight	date_of_appointment	explain_reason	email_apt	pass_apt	medication
Edit Copy Delete	1	user_name	88686564870	Routine Checkup	jimmy	Female	30.6	2025-01-03	jimmy have allergy of perfumes	user_name@gmail.com	*****	updated by admin
Edit Copy Delete	2	rajesh	574653634	Vaccination	julle	Female	30.1	2025-01-03	no comments	rahul@gamil.com	rrrrrr	Not Inserted
Edit Copy Delete	3	shreyash	62347923	Vaccination	mani	Male	20	2025-01-03	no comments	rahul@gamil.com	rrrrrr	Not Inserted
Edit Copy Delete	4	rupesh	53453453	Operation	maxy	Female	30.1	2025-01-03	operation not done before	rahul@gamil.com	rrrrrr	Not Inserted
Edit Copy Delete	5	puja	8934873	Vaccination	jon	Male	40.5	2025-01-03	no comments	rahul@gamil.com	rrrrrr	Not Inserted
Edit Copy Delete	6	pavan	6588658	Routine Checkup	romeyo	Female	30.6	2025-01-03	just regular checkup	rahul@gamil.com	rrrrrr	Not Inserted
Edit Copy Delete	7	priya	9636923	Routine Checkup	jullet	Female	45	2025-01-03	no comments	rahul@gamil.com	rrrrrr	Not Inserted
Edit Copy Delete	8	sudha	856863	Routine Checkup	prince	Male	30.6	2025-01-03	no comments	rahul@gamil.com	rrrrrr	Not Inserted
Edit Copy Delete	9	akash	780969785	Vaccination	dolly	Female	40.2	2025-01-03	healthy dog do vaccination every months	rahul@gamil.com	rrrrrr	Not Inserted
Edit Copy Delete	10	vinayak	83279892	Vaccination	rocky	Male	30.4	2025-01-03	allergy about another dogs	rahul@gamil.com	rrrrrr	Not Inserted

- Data report of user:

Extra options

	id	name	mobile	password	email
Edit Copy Delete	1	Vikas Mhetre	55555	vvvvv	vikas@gmail.com
Edit Copy Delete	2	Rahul Telgaon	666666	rrrrrr	rahul@gamil.com
Edit Copy Delete	3	raj	77777	rrrrr	raj@gmail.com
Edit Copy Delete	4	Monya	44444	sssss	shreyashpatil@gmai.com
Edit Copy Delete	5	sudha	5665656	sssss	sudha@gmail.com
Edit Copy Delete	6	neha mam	77777	vvvvv	neha@gmail.com

- Data report of admin user:

			id	name	mobile	password	email
Edit	Copy	Delete	1	vikasadmin	55555	admin	vikasadmin@gmail.com
Edit	Copy	Delete	2	adminname	5555	admin	admin@gmail.com

Check all *With selected:* Edit Copy Delete Export

4.4 Sample program code:

```
from flask import Flask, request, jsonify, render_template, redirect, url_for, session
from flask_mysqldb import MySQL


app = Flask(__name__)
app.secret_key = '8f3bd5c6491a8e29c34e2d56ef4a4e20'

# MySQL Configuration
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = ''
app.config['MYSQL_DB'] = 'myflaskdb'

mysql = MySQL(app)

@app.route('/logout')
def logout():
    session.clear()
    return render_template("login.php")



@app.route('/create.html')
def create_html():
    return render_template("/create.html")

# Create Operation
@app.route('/create', methods=['POST'])
def create_user():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        mobile=request.form['tel']
        password=request.form['password']



        cur = mysql.connection.cursor()
        cur.execute("INSERT INTO users (name, email, mobile, password) VALUES (%s, %s, %s, %s)", (name, email, mobile,
password))
        mysql.connection.commit()
        cur.execute("SET @id := 0;")
        cur.execute("UPDATE users SET id = @id := (@id + 1);")
        mysql.connection.commit()
        cur.close()
        return render_template("/login.php")

@app.route("/update.html")
def update_html():
    return render_template("update.html")
```

```
#update user
@app.route('/update', methods=['POST'])
def update_user():
    user_id = request.form['id']  # Getting ID from the form
    name = request.form['name']
    email = request.form['email']
    cur = mysql.connection.cursor()
    cur.execute("UPDATE users SET name=%s, email=%s WHERE id=%s", (name, email, user_id))
    mysql.connection.commit()
    cur.close()
    return render_template("/index.html")


@app.route("/delete.html")
def delete_html():
    return render_template("/delete.html")

# Delete Operation (taking ID from form)
@app.route('/delete', methods=['POST'])
def delete_user():
    user_id = request.form['id']  # Getting ID from the form
    cur = mysql.connection.cursor()
    cur.execute("DELETE FROM users WHERE id=%s", (user_id,))
    mysql.connection.commit()
    cur.execute("SET @id := 0;")
    cur.execute("UPDATE users SET id = @id := (@id + 1);")
    mysql.connection.commit()
    cur.close()


    return render_template("/index.html")

@app.route('/read', methods=['GET'])
def read_users():
    # Get a cursor to interact with the database
    cur = mysql.connection.cursor()

    # Execute SQL query to fetch all users
    cur.execute("SELECT * FROM users")

    # Fetch all the records
    users = cur.fetchall()

    # Close the cursor after fetching data
    cur.close()

    # Pass the data to the template and render it
    return render_template('read.html', users=users)

@app.route("/signup.php")
def signup_php():
    return render_template("signup.php")

@app.route("/admin_signup")
def admin_signup_php():
    return render_template("admin_signup.php")

@app.route('/admin_create', methods=['POST'])
def create_admin_user():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        mobile=request.form['tel']
        password=request.form['password']



        cur = mysql.connection.cursor()
        cur.execute("INSERT INTO admin_users (name, email, mobile, password) VALUES (%s, %s, %s, %s)", (name, email, mobile,
password))
        mysql.connection.commit()
        cur.execute("SET @id := 0;")
        cur.execute("UPDATE users SET id = @id := (@id + 1);")
        mysql.connection.commit()
        cur.close()
        return render_template("/admin_login.php")
```

```python
@app.route('/admin_login')
def admin_login_php():
    user=session.get('name', 'name is not register')
    return render_template("admin_login.php", user_name=user)


@app.route("/api_login_check",methods=['POST'])
def login_check():
    if request.method == 'POST':
        # Fetch email and password from form
        email = request.form['email']
        password = request.form['password']

        # Establish database connection and execute query
        try:
            cur = mysql.connection.cursor()
            cur.execute("SELECT * FROM users WHERE email = %s AND password = %s", (email, password))
            user = cur.fetchone()

            if user:
                # Store user data in session
                session['email'] = user[4]  # Assuming 4th index is email in myflaskdb in table users
                session['password'] = user[3]  # Assuming 3rd index is password in myflaskdb in table users
                session['name']=user[1]

                # Redirect to index.html
                return render_template('/home.php',user_name=user[1])
            else:
                # Invalid credentials
                session['loginmessage'] = "Invalid UserId or Password!"
                return render_template('/login.php', login_message="Invalid UserId or Password!")
        except Exception as e:
            return jsonify({"error ": str(e)})
    else:
        return jsonify({"error": "Invalid request method!"})

@app.route("/admin_dashboard.php")
def admin_dashbord_php():
    user=session.get('name', 'name is not register')
    return render_template("admin_dashboard.php",user_name=user)



@app.route("/api_admin_check",methods=['POST'])
def admin_login_check():
    if request.method == 'POST':
        # Fetch email and password from form
        email = request.form['email']
        password = request.form['password']

        # Establish database connection and execute query
        try:
            cur = mysql.connection.cursor()
            cur.execute("SELECT * FROM admin_users WHERE email = %s AND password = %s", (email, password))
            user = cur.fetchone()

            if user:
                # Store user data in session
                session['email'] = user[4]  # Assuming 4th index is email in myflaskdb in table users
                session['password'] = user[3]  # Assuming 3rd index is password in myflaskdb in table users
                session['name']=user[1]

                # Redirect to index.html
                return render_template('/admin_dashboard.php',user_name=user[1])
            else:
                # Invalid credentials
                session['loginmessage'] = "Invalid UserId or Password!"
                return render_template('/admin_login.php', login_message="Invalid UserId or Password!")
        except Exception as e:
            return jsonify({"error ": str(e)})
    else:
        return jsonify({"error": "Invalid request method!"})


@app.route('/home.php')
def home_php():
    user=session.get('name', 'name is not register')
    return render_template("home.php",user_name=user)
```

```python
@app.route('/book_appointment.php')
def book_appointment():
    user=session.get('name', 'name is not register')
    return render_template("book_appointment.php", user_name=user)




@app.route('/our_team.php')
def our_team():
    user=session.get('name', 'name is not register')
    return render_template("our_team.php", user_name=user)


@app.route('/add_appointment', methods=['POST'])
def add_appointment():
    if request.method == 'POST':
        try:
            # Fetch appointment details
            owner_name = request.form['owner_name']
            owner_tel = request.form['owner_tel']
            reason_to_book = request.form['reason_to_book']
            dog_name = request.form['dog_name']
            dog_gender = request.form['dog_gender']
            dog_weight = request.form['dog_weight']
            date_of_appointment = request.form['date_of_appointment']
            explain_reason = request.form['explain_reason']
            email_user = session.get('email', 'Email not set')
            password_user = session.get('password', 'Password not set')
            medication = "Not Inserted"

            # Generate dynamic table name based on the date
            table_name = f"appointment_{date_of_appointment.replace('-', '_')}"

            # Database interaction
            cur = mysql.connection.cursor()

            # Check if the table exists; if not, create it dynamically
            cur.execute(f"""
                CREATE TABLE IF NOT EXISTS `{table_name}` (
                    id INT AUTO_INCREMENT PRIMARY KEY,
                    owner_name VARCHAR(255),
                    owner_tel VARCHAR(20),
                    reason_to_book TEXT,
                    dog_name VARCHAR(255),
                    dog_gender VARCHAR(10),
                    dog_weight FLOAT,
                    date_of_appointment DATE,
                    explain_reason TEXT,
                    email_apt VARCHAR(255),
                    pass_apt VARCHAR(255),
                    medication TEXT
                ) ENGINE=InnoDB;
            """)

            # Check the row count in the table
            cur.execute(f"SELECT COUNT(*) FROM `{table_name}`")
            row_count = cur.fetchone()[0]
            if row_count >= 10:
                message = "The appointment limit for this date has been reached."
                return render_template('/book_appointment.php', user_name=session.get('name', 'Name not registered'),
message=message)

            # Insert the new appointment
            cur.execute(f"""
                INSERT INTO `{table_name}`
                (owner_name, owner_tel, reason_to_book, dog_name, dog_gender, dog_weight, date_of_appointment,
explain_reason, email_apt, pass_apt, medication)
                VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
            """, (owner_name, owner_tel, reason_to_book, dog_name, dog_gender, dog_weight, date_of_appointment,
explain_reason, email_user, password_user, medication))

            # Commit the changes
            mysql.connection.commit()
            cur.close()

            # Show success message
            message = "Appointment added successfully!"
            return render_template('/book_appointment.php', user_name=session.get('name', 'Name not registered'),
message=message)
```

```python
        except Exception as e:
            # Handle any errors and return a JSON response
            message = f"An error occurred: {str(e)}"
            return render_template('/home.php', user_name=session.get('name', 'Name not registered'), message=message)
    else:
        # Return an error for invalid HTTP methods
        message = "Invalid request method!"
        return render_template('/home.php', user_name=session.get('name', 'Name not registered'), message=message)


@app.route("/appointment_user.php")
def history_appointment_user():
    return render_template("appointment_user.php")


@app.route("/history_appointment_user")
def api_history_appointment_user():
    try:
        # Get email and password from session
        email = session.get('email', 'Email not set')
        password = session.get('password', 'Password not set')

        if not email or not password:
            return jsonify({"error": "Email or password not set in session"}), 400

        # Get a cursor to interact with the database
        cur = mysql.connection.cursor()
        mydb = "myflaskdb"

        # Get the list of all tables excluding 'users' and 'admin_user'
        cur.execute("""
            SELECT table_name
            FROM information_schema.tables
            WHERE table_schema = %s
            AND table_name NOT IN ('users', 'admin_user')
        """, (mydb,))

        tables = cur.fetchall()

        all_data = []

        # Loop through all tables
        for table in tables:
            table_name = table[0]

            if table_name.isidentifier():
                # Ensure email and password are valid columns in the table
                cur.execute(f"DESCRIBE `{table_name}`")
                columns = [desc[0] for desc in cur.fetchall()]

                if "email_apt" in columns and "pass_apt" in columns:
                    query = f"SELECT * FROM `{table_name}` WHERE email_apt= %s AND pass_apt= %s"
                    cur.execute(query, (email, password))
                    rows = cur.fetchall()

                    # Add each row with the table name and column headers
                    if rows:
                        for row in rows:
                            row_data = dict(zip(columns, row))
                            row_data["table_name"] = table_name  # Add table name to distinguish from other tables
                            all_data.append(row_data)

        cur.close()

        # Pass data to the template for rendering
        user = session.get('name', 'name is not register')
        return render_template('spacific_users_appointments.php', users=all_data, user_name=user)

    except Exception as e:
        return jsonify({"error": str(e)}), 500





@app.route('/spacific_users_appointments.html')
def spacific_users_appointments():
    user=session.get('name', 'name is not register')
    return render_template("spacific_users_appointments.php", user_name=user)
```

```
@app.route('/add_medication.php')
def update_spacific_user_appoitment_php():
    user=session.get('name', 'name is not register')
    return render_template("add_medication.php", user_name=user)


@app.route('/add_medication', methods=['POST'])
def update_spacific_user_appoitment():
    keys = request.form['key']  # Getting ID from the form
    date = request.form['date']
    medication = request.form['medication']

    # Format the table name dynamically based on the date
    table_name = f"appointment_{date.replace('-', '_')}"

    # Get a cursor to interact with the database
    cur = mysql.connection.cursor()

    try:
        # Execute the update query dynamically using the table name
        cur.execute(f"UPDATE `{table_name}` SET date_of_appointment=%s, medication=%s WHERE id=%s", (date, medication,
keys))
        mysql.connection.commit()

    except Exception as e:
        # Handle any potential errors
        mysql.connection.rollback()
        return jsonify({"error": str(e)}), 500

    finally:
        cur.close()

    user = session.get('name', 'name is not register')
    return render_template("add_medication.php", user_name=user)


@app.route('/to_add_medication_display_appointment_update')
def to_add_medication_display_appointment_update():

    email_specific = session.get('email', 'Email not set')
    password_spacific = session.get('password', 'Password not set')
    cur = mysql.connection.cursor()
    cur.execute("SELECT * FROM appointment_master WHERE email_apt = %s AND pass_apt = %s", (email_specific,
password_spacific))
    users = cur.fetchall()
    cur.close()
    user=session.get('name', 'name is not register')
    return render_template('add_medication.php',user_name=user, users=users)

@app.route("/search", methods=["POST"])
def search():
    query = request.form.get("query")  # Retrieve the search query from the form
    if query:
        # Example: Search logic
        if query.lower() == "book appointment" or query.lower()=="book" or query.lower()=="appointment book" or
query.lower()=="book appointments":
            user=session.get('name', 'name is not register')
            return render_template("book_appointment.php", user_name=user)

        elif query.lower() == "add medication" or query.lower()=="medication add" or query.lower()=="medication":
                user=session.get('name', 'name is not register')
                return render_template("add_medication.php",user_name=user)

        elif query.lower() == "team" or query.lower()=="doctors" or query.lower()=="assistants" or query.lower()=="doctor":
            user=session.get('name', 'name is not register')
            return render_template("our_team.php", user_name=user)

        elif query.lower() == "blogs" or query.lower()=="interesting blogs" or query.lower()=="blog" or
query.lower()=="anything to read":
            user=session.get('name', 'name is not register')
            return render_template("blogs.php", user_name=user)

        elif query.lower() == "history" or query.lower()=="my appointments history" or query.lower()=="appointments history"
or query.lower()=="appointments" or query.lower()=="appointment":
            user=session.get('name', 'name is not register')
            email_specific = session.get('email', 'Email not set')
            password_spacific = session.get('password', 'Password not set')
            cur = mysql.connection.cursor()
            cur.execute("SELECT * FROM appointment_master WHERE email_apt = %s AND pass_apt = %s", (email_specific,
password_spacific))
            users = cur.fetchall()

            return render_template("spacific_users_appointments.php",users=users ,user_name=user)
```

```
        elif query.lower() == "home" or query.lower()=="main page" or query.lower()=="home page":
            user=session.get('name', 'name is not register')
            return render_template("home.php", user_name=user)

        else:
            user=session.get('name', 'name is not register')
            not_searched="May you misspell"
            return render_template("home.php",user_name=user, not_searched=not_searched)
    else:
        user=session.get('name', 'name is not register')
        return render_template("home.php",user_name=user)



#all blog post endpoints
@app.route('/blog_1_php')
def blog_one():
    user=session.get('name', 'name is not register')
    return render_template('blog_1.php', user_name=user)

@app.route('/blog_2_php')
def blog_two():
    user=session.get('name', 'name is not register')
    return render_template('blog_2.php', user_name=user)

@app.route('/blog_3_php')
def blog_three():
    user=session.get('name', 'name is not register')
    return render_template('blog_3.php', user_name=user)

@app.route('/blog_4_php')
def blog_():
    user=session.get('name', 'name is not register')
    return render_template('blog_4.php', user_name=user)

@app.route('/blogs.php')
def blogs():
    user=session.get('name', 'name is not register')
    return render_template("blogs.php", user_name=user)

@app.route('/admin_history_appointment')
def admin_history_appointment():
    user=session.get('name', 'name is not register')
    return render_template("admin_history_appointment.php", user_name=user)

@app.route("/api_admin_history_appointment")
def api_admin_history_appointment():
    user=session.get('name', 'name is not register')
    return render_template('admin_history_appointment.php', user_name=user)


@app.route("/admin_fatch_appointment_as_given_date", methods=["POST"])
def api_admin_according_to_date_history_appointment():
    try:
        email_specific = session.get('email', 'Email not set')
        password_spacific = session.get('password', 'Password not set')

        # Get the date from the form
        date = request.form.get('date_to_see_appointment')  # Use .get() to avoid KeyError

        if not date:
            return jsonify({"error": "Date is required"}), 400

        # Validate and construct the dynamic table name
        table_name = f"appointment_{date.replace('-', '_')}"
        if not table_name.isidentifier():
            return jsonify({"error": "Invalid table name"}), 400

        # Get a cursor to interact with the database
        cur = mysql.connection.cursor()

        # Safely construct the SQL query for dynamic table name
        query = f"SELECT * FROM `{table_name}`"
        cur.execute(query)

        # Fetch all the records
        users = cur.fetchall()



        # Close the cursor after fetching data
        cur.close()
```

```
        user = session.get('name', 'name is not registered')

        # Render the template with the fetched data
        return render_template('admin_history_appointment.php', users=users, user_name=user)

    except Exception as e:
        # Handle any unexpected errors
        return jsonify({"error": str(e)}), 500


@app.route('/admin_add_medication.php')
def admin_add_medication():
    user=session.get('name', 'name is not register')
    return render_template("admin_add_medication.php",user_name=user)


@app.route('/api_admin_add_medication', methods=['POST'])
def api_admin_update_spacific_user_appoitment():
    keys = request.form['key']  # Getting ID from the form
    date = request.form['date']
    medication = request.form['medication']

    # Format the table name dynamically based on the date
    table_name = f"appointment_{date.replace('-', '_')}"

    # Get a cursor to interact with the database
    cur = mysql.connection.cursor()

    try:
        # Execute the update query dynamically using the table name
        cur.execute(f"UPDATE `{table_name}` SET date_of_appointment=%s, medication=%s WHERE id=%s", (date, medication,
keys))
        mysql.connection.commit()

    except Exception as e:
        # Handle any potential errors
        mysql.connection.rollback()
        return jsonify({"error": str(e)}), 500

    finally:
        cur.close()

    user=session.get('name', 'name is not register')
    message = "Prescription added successfully!"
    return render_template("admin_add_medication.php",user_name=user, message=message)

@app.route('/admin_to_add_medication_display_appointment_update')
def admin_to_add_medication_display_appointment_update():

    cur = mysql.connection.cursor()
    cur.execute("SELECT * FROM appointment_master ")
    users = cur.fetchall()
    cur.close()
    user=session.get('name', 'name is not register')
    return render_template('admin_add_medication.php',user_name=user, users=users)


@app.route('/')
def index():
    return render_template('/login.php')  # Ensure you create a template for this

if __name__ == '__main__':
    app.run(debug=True)
```

- Book Appointment Sample code:

```
<!DOCTYPE html>
<html lang="en">
<head>



    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="stylesheet" href="{{ url_for('static', filename='signup_css.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='book_appointment_css.css') }}">
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css">
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Josefin+Sans&display=swap" rel="stylesheet">
    <link rel="stylesheet"  href="css/style.css">
    <link rel="icon"  href="images/icon.png">
    <title>Paw Appointment Booking</title>









</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark" >
        <div class="container-fluid">
            <a class="navbar-brand" href="#" aria-disabled="true" ><label style="color:rgb(160, 231, 245)">Paw Connect
</label></a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent"
aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarSupportedContent">
                <ul class="navbar-nav me-auto mb-2 mb-lg-0">
                    <li class="nav-item">
                        <a class="nav-link active" aria-current="page" href="/home.php"><label
style="color:gray">Home</label></a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="/book_appointment.php"><label style="color:gray">Book Appointment
</label></a>
                    </li>









                    <li class="nav-item dropdown">
                        <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown" role="button" data-bs-
toggle="dropdown" aria-expanded="false"><label style="color:rgb(160, 231, 245)"> Know More </label></a>
                        <ul class="dropdown-menu" aria-labelledby="navbarDropdown">
                            <li><a class="dropdown-item" href="/blogs.php">Interesting Blogs </a></li>
                            <li><a class="dropdown-item" href="/our_team.php">Meet Our Team</a></li>
                            <li><hr class="dropdown-divider"></li>
                            <li><a class="dropdown-item" href="/history_appointment_user">See your appointments</a></li>
                            <li><a class="dropdown-item" href="/add_medication.php">Add Medication</a></li>
                        </ul>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="/" tabindex="-1" aria-disabled="true"><label style="color:gray">Log
out</label></a>
                    </li>

                        <li class="nav-item float-right">
                            <a class="nav-link disabled" href="#" tabindex="-1" aria-disabled="true"><label
style="color:rgb(160, 231, 245)">{{ user_name }}</label></a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link" href="/book_appointment.php"><label
style="color:gray">_________________</label></a>
                        </li>
                    </ul>
```

```
                    <form class="d-flex" action="/search" method="POST">
                        <input class="form-control me-2" type="search" name="query" style="background-color: rgb(160, 231,
245);" placeholder="Search" aria-label="Search">
                        <button class="btn btn-outline-info" type="submit">Search</button>
                    </form>

            </div>
        </div>
    </nav>


    <!-- Bootstrap Bundle with Popper (JS) -->
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

 <script src="https://cdn.jsdelivr.net/npm/jquery@3.6.4/dist/jquery.slim.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/popper.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/js/bootstrap.bundle.min.js"></script>
    <!-- Popup Container -->
    <div class="popup-container" id="popup">
        <div class="popup-content">
            <button class="close-btn" onclick="closePopup()">×</button>
            <p id="popup-message"></p>
        </div>
    </div>

    <!-- JavaScript for Popup -->
    <script>
        // Show the popup if a message is passed
        const message = "{{ message|safe }}";
        if (message) {
            const popup = document.getElementById('popup');
            const popupMessage = document.getElementById('popup-message');
            popupMessage.textContent = message; // Set the message text
            popup.style.display = 'flex'; // Show the popup
        }

        // Close the popup
        function closePopup() {
            const popup = document.getElementById('popup');
            popup.style.display = 'none'; // Hide the popup
        }
    </script>


<section class="my-2 container con" >
    <div  class="py-5">
        <h2 class=" p1-2 display-5"><label style="color:rgb(160, 231, 245)"> | </label><label style="color:black">  Book
Appointment </label> </h2>
    </div>
    <div class=" w-50 m-auto">
    <form action="/add_appointment" method="post">
            <div class="form-group ">
            <input type="text"  id="name"     name="owner_name"      class="fadeIn second custom_placeholder"
placeholder="Enter Owner Name" required>
            <div class="invalid-feedback">Please fill out this field.</div>
            </div>
            <div class="form-group ">
                <input type="text"  id="tel"      name="owner_tel"    class="fadeIn second custom_placeholder"
placeholder="Mobile Number" required>
            </div>
            <div class="form-group " >
            <select class="form-select  fadeIn second" name="reason_to_book" required>
                <option selected value="Routine Checkup" style="color:rgb(112, 78, 53); font-weight: 500;" ><b>Routine
Checkup</b></option>
                <option value="Vaccination " style="color:rgb(112, 78, 53); font-weight: 500;"><b>Vaccination</b></option>
                <option value="Operation " style="color:rgb(112, 78, 53); font-weight: 500;"><b>Operation</b></option>
                <option value="Emergency" style="color:red;font-weight: 600;">Emergency (we take pre-precaution before came
to clinic)</option>
            </select>


            </div>
```

```
            <div class="form-group ">
            <input type="text" id="email"    name="dog_name"     class="fadeIn second custom_placeholder" placeholder="Enter
Pet Name" required>
            <div class="invalid-feedback">Please fill out this field.</div>
            </div>
            <div class="form-group fadeIn second " style="color:rgb(112, 78, 53)">
                <label style="background-color: #e1ddddbb;">Dog Gender :</label>
                <input type="radio" id="html" name="dog_gender" value="Male">
                <label for="html">Male</label>
                <input type="radio" id="css" name="dog_gender" value="Female">
                <label for="css">Female</label>
            </div>
            <div class="form-group ">
                <input type="text"  class="fadeIn second custom_placeholder" id="password"
name="dog_weight"  placeholder="Enter weight"></br>
            </div>
            <div class="form-group fadeIn second " style="color:rgb(112, 78, 53)">
                <label style="background-color: #e1ddddbb;">Enter date of Appoitmnet : </label>
                <input type="date"  placeholder="Enter Date of Appointment " class="custom_placeholder"
name="date_of_appointment"></br>
            </div>





            <div class="form-group fadeIn second " style="color:rgb(112, 78, 53)">
                <label style="background-color: #e1ddddbb;"> Comment </label>
                <textarea name="explain_reason" class="form-control" style="color:rgb(112, 78, 53)" placeholder="Give more
tips for your pet precaution (If emergency must fill this section!) "></textarea>
           </div>

            <div class="form-group "><br>
                <input type="submit" class="fadeIn fourth " value="Book Appoitment Now">
            </div>
        <div class="mt-3">
            <p class="fadeIn fourth mb-0 text-muted color_font">Don't have an account?</p>
            <a href="/signup.php"><div class="fadeIn fourth " id="log_in_button">Sign up</div></a>
        </div>
    </form>


</section>

</body>
</html>
```

- User Dashboard sample code:

```
<!DOCTYPE html>
<html>
<head>

<meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- google fonts -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Oswald:wght@200..700&family=PT+Serif:ital,wght@0,400;0,700;1,400;1,700&family
=Roboto+Mono:ital,wght@0,100..700;1,100..700&display=swap" rel="stylesheet">
    <!-- Google fonts end -->

    <link rel="stylesheet" href="{{ url_for('static', filename='add_medication_css.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='signup_css.css') }}">

    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css">
  <link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Josefin+Sans&display=swap" rel="stylesheet">
  <link rel="stylesheet"  href="css/style.css">
```

```
  <link href="//maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/bootstrap.min.css" rel="stylesheet" id="bootstrap-css">
    <link rel="stylesheet" href="{{ url_for('static', filename='spacific_user_appointments_css.css') }}">

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-
rbsA2VBKQ0HRfSuC5I+4/yekEVnmIQLA4WbVZY72qUk5zK3n9NT1BsWuP9W2J+0U" crossorigin="anonymous">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.1/dist/css/bootstrap.min.css" rel="stylesheet" >
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.1/dist/js/bootstrap.bundle.min.js" rel="stylesheet" >
  <link href="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.2.1/jquery.min.js" rel="stylesheet" >
  <link href="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.2.1/jquery.min.js" rel="stylesheet" >
  <link href="https://use.fontawesome.com/releases/v5.7.2/css/all.css" rel="stylesheet" >
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">

    <title>Spacific_user_php</title>
</head>
<body>




<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
            <div class="container-fluid">
                <a class="navbar-brand " href="#" aria-disabled="true"><label style="color:rgb(160, 231, 245)">Paw Connect
</label></a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle
navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>
                <div class="collapse navbar-collapse" id="navbarSupportedContent">
                    <ul class="navbar-nav me-auto mb-2 mb-lg-0">
                        <li class="nav-item">
                            <a class="nav-link active" aria-current="page" href="/admin_dashboard.php"><label
style="color:gray">Home</label></a>
                        </li>
                        <li class="nav-item dropdown">
                            <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown" role="button" data-bs-
toggle="dropdown" aria-expanded="false"><label style="color:rgb(160, 231, 245)"> Know More </label></a>
                            <ul class="dropdown-menu" aria-labelledby="navbarDropdown">
                                <li><a class="dropdown-item" href="/api_admin_history_appointment">See your
appointments</a></li>
                                <li><a class="dropdown-item" href="/admin_add_medication.php">Add Medication</a></li>
                            </ul>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link" href="#" tabindex="-1" aria-disabled="true"><label style="color:rgb(160,
231, 245)">{{ user_name }}</label></a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link" href="/" tabindex="-1" aria-disabled="true"><label style="color:gray">Log
out</label></a>
                        </li>
                    </ul>
                </div>
            </div>
        </nav>




    <!-- Bootstrap Bundle with Popper (JS) -->
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>



<div class=" box_2_medication_css d-flex flex-column h-100">
    <div class="mt-5">
        <p class="mb-1 h-1"> Enter date to see appoitments <label class="color_font"> ! <t></label></p>
        <p class="text-muted mb-2"><label class="color_font"> | <t></label>Enter date which appointment you want to display
</p>
        <div class="d-flex flex-column ">
            <div class="d-flex align-items-center">
                <form action="/admin_fatch_appointment_as_given_date" method="POST">
                    <label style="color:rgb(112, 78, 53); background-color: #e1ddddbb; margin-left: 2em;" class="fadeIn
second ">Date (year/month/date):</label>
                    <input type="date" name="date_to_see_appointment" id="name" class="fadeIn second custom_placeholder "
required><br><br>
                    <input type="submit" class="fadeIn fourth " style="margin-left: 2em;" value="Fatch appointment according
to date">
                </form>
            </div>
     </div>
```

```
    </div>
</div>
<br>
<br>



    <!-- <h1 class="fadeIn first">All Users</h1>

    <-- Table to display the users -->
    <table border="1" class="table table-info table-striped table-hover table-bordered">
        <thead>
            <tr class="fadeIn first">
                <th>Record ID</th>
                <th>Name</th>
                <th>Owner Ph.No</th>
                <th>Reason to book</th>
                <th>Pet Name </th>
                <th>Pet Gender</th>
                <th>pet Weight </th>
                <th>Date of Appointment</th>
                <th>Description</th>
                <th>Medication</th>

            </tr>
        </thead>
        <tbody class="fadeIn second">
                <!-- {% set seen = [] %}              -->
                {% for user in users %}
                     {% if user  in users %}
                     <!-- {% set _ = seen.append(user|tojson) %} -->
                 <tr>
                    <td>{{ user[0] }}</td> <!-- user[0] is the ID -->
                   <td>{{ user[1] }}</td> <!-- user[1] is the Name -->
                   <td>{{ user[2] }}</td> <!-- user[2] is the Email -->
                   <td>{{ user[3] }}</td> <!-- user[2] is the Email -->
                   <td>{{ user[4] }}</td> <!-- user[2] is the Email -->
                   <td>{{ user[5] }}</td> <!-- user[2] is the Email -->
                   <td>{{ user[6] }}</td> <!-- user[2] is the Email -->
                   <td>{{ user[7] }}</td> <!-- user[2] is the Email -->
                   <td>{{ user[8] }}</td> <!-- user[2] is the Email -->
                   <td>{{ user[11] }}</td> <!-- user[2] is the Email -->
                </tr>
                     {% endif %}
                {% endfor %}
        </tbody>
    </table>

</body>
</html>
```

4.5 Limitations

- **Internet Dependency:** The system requires a stable internet connection for optimal functionality.
- **Limited Mobile App Integration:** Currently, the platform is web-based and lacks a dedicated mobile application.
- **Feature Expansion:** Advanced features such as automated reminders and integrated payment gateways are not yet implemented.
- **Device Compatibility:** While the system supports major web browsers, older devices with outdated browsers may face compatibility issues.
- **Customization Limits:** The system's features are predefined, and significant customization requires backend code modifications.

Bibliography

1. Flask Documentation: https://flask.palletsprojects.com/
2. PHP Official Website: https://www.php.net/
3. MySQL Documentation: https://dev.mysql.com/doc/
4. Bootstrap Documentation: https://getbootstrap.com/
5. Web Accessibility Guidelines: https://www.w3.org/WAI/